DAW_M03_T06_ACT_06_Cristian_Cantero.

Gestión de archivos con la clase File

El objetivo de este ejercicio es implementar un sistema de archivos usando la clase File. Vamos a definir un menú con creación de archivos, listado, búsqueda y borrado dentro de una carpeta que se llamará ‘archivos’.

La estructura general del programa consistirá en dividir las responabilidades de forma más correcta posible usando una Clase Funciones y otra Clase Main.
Funciones se encargará de albergar todas las funciones que crearemos y que el programa utilizará.
La Clase Main será la encargada de ejecutar el programa y sus funciones.

Primero empiezo por la Clase Funciones ya que es el cerebro del programa. A continuación voy a explicar como he programado cada una.

Función nuevoArchivo()
Esta función se encarga de crear una carpeta (si no existe) que se llamará 'archivos' y podremos crear y escribir documentos dentro.

```java
public static void nuevoArchivo(Scanner sc) throws IOException {
    String ruta = System.getProperty("user.dir");
    String sep = File.separator;
    File archivo = new File(ruta + sep + "archivos");
    if (!archivo.exists()) {
        System.out.println("Este archivo no existe");
        System.out.println("Creando archivo...");
        archivo.mkdir();
        System.out.println("Archivo creado en: " + archivo.getAbsolutePath());
    } else {
        System.out.println("Esta carpeta existe en: " + archivo.getAbsolutePath());
    }
    System.out.println("Introduce el nombre del archivo.txt que quieres crear: ");
    String nombre = sc.nextLine();

    File texto = new File(archivo, nombre);
    if (!texto.exists()) {
        texto.createNewFile();
        System.out.println("Creando hoja de registro en: " + texto.getAbsolutePath());
    }
```

nuevoArchivo() recibirá por parámetros una variable Scanner para que el usuario pueda introducir datos por consola. Más adelante, cuando siga explicando el código, aclararé porque he usado throws IOException como manejo de errores.

Le damos las instrucciones para crear la ruta donde se creará la carpeta. En este caso *System.getProperty("user.dir")*; le indica al programa que la ruta estará en la raiz del proyecto.
File.separator será completamente ajeno al tipo de sistema operativo que esté utilizando este programa, así nos aseguramos que las rutas se generan correctamente ya sea en windows, macOS o Linux.
Toda esta ruta queda almacenada dentro de un obeto File que de nombre le he puesto archivo. Lo siguiente es cerciorarnos que esta carpeta exista o no. Si archivo no existe dará un mensaje avisandolo, otro diciendo que crea la carpeta y ,seguidamente, creará la carpeta en la raiz del proyecto. Acaba este proceso lanzando un mensaje de la correcta creación de la carpeta y la ruta donde se encuentra.
Por contra, si la carpeta SI existiera, se da el mensaje de confirmación y la ruta correspondiente.
Después nos pide que introduzcamos el nombre del documento que queremos ingresar dentro de la carpeta. El programa guarda el nombre en una variable.

Declaro un objeto File para manejar el documento que se almacenara dentro de la carpeta. File texto sera un nuevo objeto File cuya ruta será archivo y nombre.
Si dentro de la carpeta ‘archivos’ no existe ninguna coincidencia con otro documente se creará el documento usando createNewFile(). Aquí es donde voy a explicar el uso de throws IOException. El método createNewFile() puede lanzar muchos tipos execepciones (permisos, disco lleno, carpeta inexistente, archivo bloqueado, sistema de archivos corrupto…) Por esta razon Java me obliga a manejar el error. Podría haber hecho un try/catch pero opté por esta opción. Lo interesante/curioso es que el error en el IDE solo iluminaba en rojo los parentesis del método. Lo que más he aprendido de esto es que Java obliga al programador a reconocer los errores posibles. No permite ignorarlos.
Finalmente se crea el documento, se da el mensaje de confirmación y la ruta del mismo.

```
    //escritura del .txt
    System.out.println("Escribe lo que quieras en tu archivo " + texto);
    String escribir = sc.nextLine();
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(texto, true))) {
        bw.write(escribir);
        bw.newLine();
    } catch (IOException e) {
        System.out.println("Error en la escritura");
    }
    System.out.println("Esto es lo que has escrito en tu archivo " + nombre);
    System.out.println(escribir);

}
```

Ahora procedemos a la escritura del documento.
Primero damos el mensaje para que el user pueda escribir en el documeto. Guardamos el texto en una variable. Y vamos con el objeto File BufferedWriter para estampar en el documento el texto que está guardado en la variable. Controlamos la excepcion y finalmente si todo esta correcto damos el mensaje mostrandole al user que es lo que ha escrito.

Función listarArchivos()

Esta función se encargará de mostrar todos los documentos .txt que hay dentro de la carpeta ‘archivos’ numerados y por nombre.

```java
public static File[] listarArchivos() {
    String ruta = System.getProperty("user.dir");
    String sep = File.separator;
    File archivo = new File(ruta + sep + "archivos");
    if (!archivo.exists()) {
        System.out.println("Esta carpeta no existe");
        archivo.mkdir();
    }
    File[] contenido = archivo.listFiles();

    if (contenido == null) {
        return new File[0];

    }
    int contador = 0; //este primer contador es para saber cuantos .txt válidos hay
    for (File f : contenido) {
        if (f.isFile() && f.getName().endsWith(".txt")) {
            contador++;
        }
    }
    File[] resultado = new File[contador];
    int indice = 0; // este índice sirve para saber en qué posición (hueco) del array []resultado estoy guardando.
    for (File f : contenido) {
        if (f.isFile() && f.getName().endsWith(".txt")) {
            resultado[indice] = f;
            indice++;
        }
    }
```

Como esta función no es void, lo que hace es devolver un valor. En este caso será un array de objetos File (de ahí la firma File[]). Ese valor lo gestionaré a través del método Main. De esta manera divido las responsabilidades dentro del programa. Tengo que decir que al principio (y por inexperiencia/inercia) intenté gestionarlo todo desde la misma función pero como resultado la consola imprimia por duplicado el contenido… esta fue la manera como me enteré que dividir responsabilidades en este tipo de trabajos era importante (aunque no soy capaz técnicamente de explicar el porqué se duplicaba entendí donde estaba el asunto).

La función empieza creando las rutas de carpeta exactamente igual que en la funcion anterior.

Para listar los documentos dentro de la carpeta usaré un método propio de la clase File de nombre listFiles(); Este método devuelve una array de objetos File donde cada objeto representa un archivo o una subcarpeta. Este resultado lo guardo en un array de File que lo llamaré contenido.

Para controlar que este array de File no esté vacío (null) uso un condicional que devolvera un array de tamaño 0 en caso de que ocurra. Esto lo hago para evitar un NullPointerException. Es recomendable porque si nos quedasemos con el null que devolvería el array vacío proveniente de listFiles(), y luego quisieramos recuperar por ejmplo el nombre con getName(), simplemente el programa explotaría con este NullPointerException.
Como curiosidad agregaré que este patrón se le llama “return empty instead of null”. Devuelve vacío en vez de nulo (más o menos).

Lo siguiente que me interesa es que los documentos dentro de la carpeta sean .txt y no otro tipo. Para esto voy a recorrer todos los elementos de la carpeta usando un for/each y si acaban en .txt los sumaré al contador. Esto se conoce como contar archivos válidos. Este bucle no guarda nada, solo saber cuantos .txt hay dentro de la carpeta. Para este bloque he usado método propio de File como .isFile() - para saber si es un archivo/ .getName() - para obtener el nombre/ y endsWith() - para aclarar que el archivo tiene que acabar en .txt

Después voy a guardarlos en un array que tiene que tener el tamaño correcto. Ya en el siguiente for si voy a guardar los archivos. Para eso necesito saber en que posicion del array escribirlo. Declaro una variable indice. Y el bucle for/each crea una variable auxiliar f que recorre el array contenido. Si cumple las condiciones (acaba en .txt) lo guarda.

La necesidad de utilizar dos bucles radica en que en Java los arrays no pueden cambiar de tamaño. Este patrón se llama “count+fill pattern” y se usaba mucho antes de que Java tuviera ArrayList tan extendido.

Finalmente la función devuelve un resultado que será manejado en la Clase Main.

Función buscarArchivo()

Esta función se encargará de buscar el documento que el usuario desee.

```java
public static void buscarArchivo(Scanner sc) throws IOException {
    boolean encontrado = false;
    File[] archivos = listarArchivos();
    if (archivos.length == 0) {
        System.out.println("No hay archivos disponibles");
        return;
    }

    System.out.println("Introduce el nombre del archivo que quieres ver.");
    String nombreArchivo = sc.nextLine();
    for (File f : archivos) {

        if (nombreArchivo.equalsIgnoreCase(f.getName())) {
            encontrado = true;
            try (BufferedReader br = new BufferedReader(new FileReader(f))) {
                String linea;
                while ((linea = br.readLine()) != null) {
                    System.out.println(linea);
                }

            } catch (IOException e) {
                System.out.println("Error en la lectura motrar ARchivo" + e.getMessage());
            }
            break;
        }

    }
    if (!encontrado) {
        System.out.println("No existe un documento con este nombre.");
    }

}
```

funciones > listarArchivos > for (File f : contenido > if (f.isFile(&& f.getName(.endsWith(".txt" >

buscarArchivo() recibe como parámetros la variable de Scanner sc porque el usuario deberá introducir por consola el nombre del archivo que quiere buscar.
Declaro ‘encontrado’ como variable que me sirve de “bandera booleana” para controlar acciones como haber encontrado el documento y poder proceder con la siguiente acción en consecuencia.
Vuelvo a usar el valiosísimo método listarArchivos() para encontrar el documento deseado.
Compruebo que la carpeta tenga archivos antes. Si no los hay salta un mensaje avisando de que la carpeta está vacía. Y sale al menu principal.

Una vez hecha esa comprobación el programa solicita al usuario el nombre del documento que quiere buscar. Una vez introducido se guarda en una variable de tipo String de nombre ‘nombreArchivo’.

Como la intención es solo leer los datos y no quiero controlar ningún índice, uso un bucle for/each para recorrer ‘archivos’ que a su vez utiliza listarArchivos() tal y como está declarado al principio de la función:
File[] archivos = listarArchivos();

Y empezamos con las condiciones que se tienen que cumplir para que lea el interior del documento.

Si el nombre introducido por el usuario es igual al nombre que toca en la iteración sin importar mayúsculas o minúsculas declara la bandera booleana como true, lee la línea y la imprime por pantalla. Todo en ese orden.
Para las condiciones del if utilicé el método propio de la Clase File getName(). Y BufferedReader para leer el contenido.
Si hubiera cualquier fallo en la lectura del documento saltaría un mensaje de error. Para que el bucle se detenga cuando haya encontrado el documento usaré el break.
La última línea de la función hace saltar un mensaje en caso de que no se encontrase coincidencia con el nombre introducido por el usuario.

Función renombrarTxt()

Esta función se encargará de cambiar el nombre del documento que el usuario desee cambiar.

```java
public static void renombrarTxt(Scanner sc) {
    boolean encontrado = false;

    File[] archivos = listarArchivos();

    if (archivos.length == 0) {
        System.out.println("Esta carpeta no tiene archivos");
        return;
    }

    System.out.println("¿Qué archivo quieres renombrar?");
    String archivoRenombrar = sc.nextLine();

    for (File f : archivos) {
        if (archivoRenombrar.equalsIgnoreCase(f.getName())) {
            encontrado = true;

            System.out.println("Escribe el nombre nuevo");
            String nombreNuevo = sc.nextLine();
            String carpeta = f.getParent();
            File nuevo = new File(carpeta, nombreNuevo);
            if (nuevo.exists()) {
                System.out.println("Este nombre ya existe en otro .txt");
                return;

            } else {
                f.renameTo(nuevo);
                break;
            }
        }
    }
}
```

Como parámetros recibe una variable de Scanner sc ya que el usuario tendrá que introducir por consola el nombre del archivo que quiere renombrar.
Declaro una ‘bandera booleana’ de nombre ‘encontrado’ para manejar acciones.
La maravillosa función re-re-reutilizable listarArchivos() vuelve a la escena y la guardamos en un array de File que, de forma original, lo llamaré ‘archivos’.
Se hace la comprobación de seguridad para cerciorarse de que la carpeta no esté vacía. Si lo estuviera: mensaje y al menú principal.

El sistema le pide al usuario que introduzca el nombre del documento que quiere renombrar. Y lo guarda en una variable String llamada ‘archivoRenombrar’.
Vuelvo al bucle for each. No quiero controlar ningún índice, solo leer los datos así que lo veo como la mejor opción otra vez.
Si encuentra el documento porque coincide en el nombre (hará la comparación obviando mayúsculas y minusculas, cortesia de .equalsIgnoreCase()) levanta la bandera booleana a true y le pide al usuario que escriba el nuevo nombre para el documento. Lo guarda en una variable String llamada ‘nombreNuevo’. Después utilizo el método getParent() y así obtengo la ruta de la carpeta donde se encuentra el archivo.
La razón de usar el método getParent() es para asegurarme que no muevo el archivo de la carpeta sin querer. Este método devuelve la ruta de la carpeta que contiene el archivo. El programa quiere renombrar el archivo, pero sin cambiarlo de carpeta. Por eso trabajamos con File, getParent() nos devuelve la representación de la ruta donde se encuentra nuestro documento.
Declaro un nuevo File que, originalmente, le llamo ‘nuevo’ y contendrá la representacion de la ruta con el nombre ya cambiado.
Hacemos una comprobación de que ese nombre no exista ya dentro de la carpeta. Y si es así, usando el método renameTo() le asignamos el nuevo nombre al archivo. Con un break salimos del bucle.

Funcion borrarArchivo()

Esta función se encargará de borrar el documento dentro de la carpeta ‘archivos’ que el usuario desee.

```java
public static void borrarArchivo(Scanner sc) {
    boolean encontrado = false;
    File[] archivos = listarArchivos();

    if (archivos.length == 0) {
        System.out.println("Esta carpeta está vacía");
        return;
    }

    System.out.println("Introduce el archivo que quieres borrar");
    String archivoBorrar = sc.nextLine();

    for (File f : archivos) {
        if (archivoBorrar.equalsIgnoreCase(f.getName())) {

            boolean borrado = f.delete(); //guarda el dato para usarlo
            if (borrado) {
                encontrado = true;
                System.out.println("Archivo eliminado correctamente");
                break;
            } else {
                System.out.println("No se pudo eliminar el archivo");
                encontrado = true;
                break;

            }

        }

    }
```

Como parámetro recibe una variable de Scanner sc para que el usuario introduzca el nombre del documento que quiere borrar. Levanto una bandera booleana para controlar acciones. Y, como no, listarArchivos(). Es decir, reutilización de código muy gustosa. Creo que la he utilizado en todas las funciones. La guardo para usarla en un array de File que llamaré ‘archivos’. Hago una comprobación de seguridad para saber que la carpeta no está vacía. Y si no lo está (vacía), continuamos con la ejecución del programa.

El programa pregunta al usuario que introduzca el nombre del documento que quiere borrar. Y lo guarda en una variable String de nombre ‘archivoBorrar’.

Como solo me interesa recorrer ‘archivos’ no me hace falta ningún índice, así que vuelvo a optar por el bucle for/each para recorrer el contenido de la carpeta.
Si el nombre almacenado en ‘archivoBorrar’ coincide (no importa si mayuscula o minúscula gracias a equalsIgnoreCase) con el nombre del documento (obtenido gracias a f.getName()) sucede lo siguiente:
Declaro una variable booleana de nombre ‘borrado’ y usaré el método delete(). Este método de la Clase File intenta eliminar el archivo del sistema. La razon de guardar su acción en una variable booleana es porque este método devuelve true o false. Así que f.delete() guarda el resultado true o false en la variable ‘borrado’. De esta manera decido que mensaje mostrar en caso de que sea una cosa u otra (true o false).
Si ‘borrado’ es true levantamos la bandera booleana a true y mostramos el mensaje correspondiente. Acaba la acción con un break para salir del bucle. Si no, se muestra el mensaje correspondiente, volvemos a levantar la bandera a true igualmente y despues de dar el mensaje salimos del bucle con un break.
<u>Parentesis explicativo de la bandera booleana doblemente true en el if:</u>
Es un concepto que todavía me cuesta entender y, sobre todo, me cuesta saber cuando se puede utilizar este ‘patron’. La variable booleana ‘encontrado’ no indica si el archivo se ha borrado, sino si el archivo existe y ha sido localizado durante la búsqueda. Por eso se establece como true independientemente de si la operacion delete() tiene éxito o no. De hecho podría haberlo simplificado declarando ‘encontrado’ en true una línea arriba del if.

Una vez salimos del for se hace una comprobación:

```
}
if (!encontrado) {
    System.out.println("No existe un archivo con ese nombre.");

}
```

Si no ha habido coincidencia la variable booleana ‘encontrado’ nunca habrá cambiado a true. Por lo tanto no se habrá encontrado el nombre del archivo introducido por el usuario porque no habrá habido coincidencia.

Notas y conclusiones finales

Soy consciente de que en este proyecto he tenido un “cruce” importante con los nombres. Por ejemplo en el caso de los archivos y documentos. He tenido cruzados los nombres carpeta y archivo creyendo que en una carpeta había archivos para que se entienda. De todos modos, he sido consciente de lo que iba dentro y fuera de la carpeta sobre la que trabajaba. Pero creo que he cruzado los nombres. Ahora leyendo el código me parece “raro”. Pero tomo nota totalmente y ya lo tengo claro. Un archivo es una carpeta y dentro hay subcarpetas y documentos.
La razón por la cual no he cambiado los nombres es que ya había invertido muchas horas, rellenado mucho el pdf y el programa funcionaba como pedía el enunciado.
He invertido más de 10 horas para resolver este ejercicio. He disfrutado mucho trabajando en el y me gustaría desarrollar herramientas de estudio que me ayuden a interiorizar como programar, como tener más soltura. Estoy “agarrotao” programando. No me veo resolviendo otro ejercicio como este sin consultar lo que he hecho aquí.
Conclusión final: **Hay que estudiar mucho**.

Bibliografía:
Documentación consulatada : Clases de Linkia, imprimibles de Linkia.
Herramientas utilizadas para el aprendizaje : NoteBook LM, Chat GPT.